

Task 278: Veterinary Clinic A veterinary clinic treats animals brought in by their owners. Each owner may bring in several pets, and every pet has its own medical history that grows over time and should never be replaced wholesale, only added to. During a visit, a vet performs an examination that records a diagnosis and a set of vital signs; if a vital sign falls outside a safe range, the examination should be able to flag itself as urgent. Each examination can result in one or more treatments being prescribed, and every treatment has a cost that contributes to a running total for the visit — a total that should always be calculated, never stored and edited directly. A pet also belongs to a species with its own care requirements, and a pet's age should be derived from its date of birth rather than set manually. Think about what describes an owner, a pet, an examination, and a treatment, how the visit total stays trustworthy, and how these connect to form one visit.

CLASS Clinic

ATTRIBUTES

clinicID : Number
clinicName : Text
phone : Text
location : Text
owners : List<Owner>
pets : List<Pet>

CONSTRUCTOR(n : Text, p : Text, l : Text, o : List<Owner>, petsList : List<Pet>)

SET clinicName TO n
SET phone TO p
SET location TO l
SET owners TO o
SET pets TO petsList
END CONSTRUCTOR

METHODS

addOwner(o : Owner)
ADD o TO owners
END METHOD

addPet(p : Pet)
ADD p TO pets
END METHOD

END CLASS

CLASS Pet

ATTRIBUTES

petID : Number
petName : Text
owner : Owner
DOB: Number
species : Species
medicalHistory : List<History>
visits : List<Visit>

CONSTRUCTOR(n : Text, o : Owner, dob : Number, s : Species, h : List<History>, v : List<Visit>)

SET petName TO n
SET owner TO o
SET DOB TO dob
SET species TO s
SET medicalHistory TO h
SET visits TO v

END CONSTRUCTOR

METHODS

addHistory(h : History)
 ADD h TO medicalHistory
END METHOD

addVisit(v : Visit)
 ADD v TO visits
END METHOD

getAge(currentYear : Number)
 DISPLAY currentYear - DOB
END METHOD

END CLASS

CLASS Owner

ATTRIBUTES

ownerID : Number
ownerName : Text
phone : Text
pets : List<Pet>

CONSTRUCTOR(n : Text, ph : Text, p : List<Pet>)

SET ownerName TO n
SET phone TO ph

```
        SET pets TO p
    END CONSTRUCTOR
```

```
    METHODS
```

```
        addPet(p : Pet)
            ADD p TO pets
        END METHOD
```

```
END CLASS
```

```
CLASS Species
```

```
    ATTRIBUTES
```

```
        speciesID : Number
        speciesName : Text
        careRequirements : Text
```

```
    CONSTRUCTOR(n : Text, c : Text)
        SET speciesName TO n
        SET careRequirements TO c
    END CONSTRUCTOR
```

```
    METHODS
```

```
        getCareRequirements()
            DISPLAY careRequirements
        END METHOD
```

```
END CLASS
```

```
CLASS History
```

```
    ATTRIBUTES
```

```
        historyID : Number
        date : Text
        description : Text
```

```
    CONSTRUCTOR(d : Text, desc : Text)
        SET date TO d
```

```
        SET description TO desc
    END CONSTRUCTOR
```

```
END CLASS
```

```
CLASS Vet
```

```
    ATTRIBUTES
```

```
        vetID : Number
        vetName : Text
        specialization : Text
```

```
    CONSTRUCTOR(n : Text, s : Text)
        SET vetName TO n
        SET specialization TO s
    END CONSTRUCTOR
```

```
END CLASS
```

```
CLASS VitalSign
```

```
    ATTRIBUTES
```

```
        signName : Text
        value : Number
        minSafe : Number
        maxSafe : Number
```

```
    CONSTRUCTOR(n : Text, v : Number, min : Number, max : Number)
        SET signName TO n
        SET value TO v
        SET minSafe TO min
        SET maxSafe TO max
    END CONSTRUCTOR
```

```
END CLASS
```

```
CLASS Examination
```

```
    ATTRIBUTES
```

```
        examID : Number
        diagnosis : Text
        vitalSigns : List<VitalSign>
        treatments : List<Treatment>
```

```
    CONSTRUCTOR(d : Text, v : List<VitalSign>, t : List<Treatment>)
        SET diagnosis TO d
        SET vitalSigns TO v
        SET treatments TO t
```

END CONSTRUCTOR

METHODS

addTreatment(t : Treatment)

 ADD t TO treatments

END METHOD

isUrgent()

 SET URGENT TO FALSE

 REPEAT FROM I = 0 TO SIZE OF vitalSigns -1

 IF vitalSigns[I].value LESS THAN vitalSigns[I].minSafe OR vitalSigns[I].value GREATER
 THAN vitalSigns[I].maxSafe THEN

 SET URGENT TO TRUE

 END IF

 END REPEAT

 IF URGENT EQUALS TRUE THEN

 DISPLAY "Urgent Examination"

 ELSE

 DISPLAY "Normal Examination"

 END IF

END METHOD

END CLASS

CLASS Treatment

 ATTRIBUTES

 treatmentID : Number

 treatmentName : Text

 cost : Number

 CONSTRUCTOR(n : Text, c : Number)

 SET treatmentName TO n

 SET cost TO c

 END CONSTRUCTOR

END CLASS

CLASS Visit

ATTRIBUTES

visitID : Number
pet : Pet
vet : Vet
date : Text
examination : Examination

CONSTRUCTOR(p : Pet, v : Vet, d : Text, e : Examination)

SET pet TO p
SET vet TO v
SET date TO d
SET examination TO e

END CONSTRUCTOR

METHODS

calcTotal()

SET TOTAL TO 0

REPEAT FROM I = 0 TO SIZE OF examination.treatments -1

SET TOTAL TO TOTAL + examination.treatments[I].cost

END REPEAT

RETURN TOTAL

END METHOD

END CLASS

START

BEGIN

CREATE OBJECT clinic1 OF Clinic("Muscat Pets", "999999999", "Muscat", [], [])

CREATE OBJECT owner1 OF Owner("Ali", "91234567", [])

CREATE OBJECT species1 OF Species("Cat", "Needs regular vaccination")

CREATE OBJECT pet1 OF Pet("Milo", owner1, 2020, species1, [], [])

clinic1.addOwner(owner1)

clinic1.addPet(pet1)

owner1.addPet(pet1)

CREATE OBJECT history1 OF History("2026-07-05", "First visit")

pet1.addHistory(history1)

CREATE OBJECT vet1 OF Vet("Dr. Said", "General")

CREATE OBJECT sign1 OF VitalSign("Temperature", 41, 37, 39)

CREATE OBJECT sign2 OF VitalSign("Heart Rate", 120, 60, 140)

SET signs TO LIST [sign1, sign2]

CREATE OBJECT treatment1 OF Treatment("Medicine", 10)

CREATE OBJECT treatment2 OF Treatment("Injection", 15)

SET treatments TO LIST [treatment1, treatment2]

CREATE OBJECT exam1 OF Examination("Fever", signs, treatments)

exam1.isUrgent()

CREATE OBJECT visit1 OF Visit(pet1, vet1, "2026-07-05", exam1)

pet1.addVisit(visit1)

visit1.calcTotal()

pet1.getAge(2026)

END

STOP

=====

Task 279: Movie Theater A cinema shows films across several screens throughout the day. Each film has its own details and is scheduled into several showtimes, and a showtime should refuse to be created if it overlaps another showtime on the same screen. When a customer books a ticket, that ticket is tied to one specific showtime and reserves a seat in the screening room — a seat that, once booked, cannot be booked again for that showtime. A screening room belongs to a particular screen with its own seating layout and a fixed capacity that no booking process should be able to exceed. A showtime should be able to report how many seats remain available at any moment, calculated rather than tracked as a separate editable number. Consider how a film, a showtime, a ticket, and a screen relate to form a complete booking, and how seat availability stays accurate.

CLASS cinema

Attributes

Id : Number
cname : TEXT
location TEXT
films : LIST<film>
Screens : LIST <Screen>

CONSTRUCTOR(c:text, l:text, f: list<film>, s:LIST <Screen>)

SET cname TO c
SET location TO l
SET films TO f
SET Screens TO s

END CONSTRUCTOR

METHODS

getCName()
 DISPLAY cname
END METHOD
setCName(Newc: TEXT)
 SET cname TO Newc
END METHOD

getLocation()
 DISPLAY location
END METHOD
setLocation(Newl: TEXT)
 SET location TO Newl
END METHOD

addFilm(f : Film)
 ADD f TO films
END METHOD

addScreen(s : Screen)


```
        ADD s TO screens
    END METHOD
```

```
END CLASS
```

```
CLASS film
```

```
    Attributes
```

```
        Id : Number
        fname: TEXT
        duration TEXT
        showtimes : LIST<showtime>
```

```
    CONSTRUCTOR(c:text, d:text, s: list<showtime >)
```

```
        set fname TO c
        set duration TO d
        set showtimes TO s
```

```
    END CONSTRUCTOR
```

```
    METHODS
```

```
        getFname()
```

```
            DISPLAY fname
```

```
    END METHOD
```

```
        setFname(Newf: TEXT)
```

```
            SET fname TO Newf
```

```
    END METHOD
```

```
        getDuration()
```

```
            DISPLAY duration
```

```
    END METHOD
```

```
        setDuration(Newd: TEXT)
```

```
            SET duration TO Newd
```

```
    END METHOD
```

```
        addShowtime(Newshowtime)
```

```
            ADD Newshowtime TO showtimes
```

```
    END METHOD
```

```
END CLASS
```

```
CLASS Screen
```

```
    Attributes
```

```
        screenID : Number
```

```
        capacity : Number
```

```
        seats : List<Seat>
```

```
        showtimes : List<Showtime>
```

```
    CONSTRUCTOR(c : Number, s : List<Seat>, show : List<Showtime>)
```

```
        SET capacity TO c
```

```
        SET seats TO s
```

```

        SET showtimes TO show
END CONSTRUCTOR
METHODS
    getCap()
        DISPLAY capacity
    END METHOD
    updateCap(cap : Number)
        SET capacity TO cap
    END METHOD
    addShowtime(newShow)
        SET OVERLAP TO FALSE
        REPEAT FROM I = 0 TO SIZE OF showtimes -1

            IF newShow.startTime LSES TAHN showtimes [ I ].endTime AND
newShow.endTime GRATER TAHN showtimes [ I ].startTime THEN
                SET OVERLAP TO TRUE
            END IF

        END REPEAT

        IF OVERLAP EQUALS TRUE
            DISPLAY "Schedule Overlap"
        ELSE
            ADD newShow TO showtimes
        END IF

    END METHOD
END CLASS

```

CLASS Showtime

Attributes

```

    film : Film
    screen : Screen
    startTime : Number
    endTime : Number
    tickets : List<Ticket>

```

```

CONSTRUCTOR(f : Film, s : Screen, st : Number, et : Number)

```

```

    SET film TO f
    SET screen TO s
    SET startTime TO st
    SET endTime TO et

```

END CONSTRUCTOR

METHODS

 seatsAvailable()

 SET remaining TO screen.capacity - SIZE OF tickets

END METHOD

 bookTicket(newTicket : Ticket)

 SET SeatAV TO FALSE

 IF SIZE OF tickets GREATER THAN OR EQUALS screen.capacity THEN

 DISPLAY "No seats available"

 END IF

 REPEAT FROM I = 0 TO SIZE OF tickets-1

 IF tickets[I].seat EQUALS newTicket.seat THEN

 SET SeatAV TO TRUE

 END IF

 END REPEAT

 IF SeatAV EQUALS TRUE THEN

 DISPLAY "Seat already booked"

 ELSE

 ADD newTicket TO tickets

 END IF

END METHOD

END CLASS

CLASS Customer

 Attributes

 customerID : Number

 name : Text

 tickets : List<Ticket>

 CONSTRUCTOR (n: Text , t:List<Ticket>)

 SET name to n

 SET tickets to t

END CONSTRUCTOR

METHODS

```
        GETName()
            DISPLAY name
        END METHODS
    END CLASS
```

CLASS Ticket

Attributes

```
    tID : Number
    showtime : Showtime
    seat : Seat
    CONSTRUCTOR (show: Showtime, se: Seat)
        SET showtime to show
        SET seat to se
    END CONSTRUCTOR
```

METHODS

```
    GETshowtime()
        DISPLAY showtime
    END METHODS
```

END CLASS

CLASS Seat

ATTRIBUTES

```
    seatID : Number
    row : Text
    number : Number
```

```
    CONSTRUCTOR( r: Text, n: Number)
        SET row TO r
        SET number TO n
    END CONSTRUCTOR
```

METHODS

```
    getSeatLabel()
        RETURN row + number
    END METHOD
```

END CLASS

START

BEGIN

CREATE OBJECT cinema1 OF Cinema("Oman Cinema", "Suhar",[], [])

CREATE OBJECT seat1 OF Seat(1, "A", 1)

CREATE OBJECT seat2 OF Seat(2, "A", 2)

CREATE OBJECT seat3 OF Seat(3, "A", 3)

SET seatList TO LIST [seat1, seat2, seat3]

CREATE OBJECT screen1 OF Screen(100, seatList, [])

cinema1.addScreen(screen1)

CREATE OBJECT film1 OF Film("Avengers", "120 min", [])

cinema1.addFilm(film1)

CREATE OBJECT showtime1 OF Showtime(film1, screen1, 10, 12)

screen1.addShowtime(showtime1)

film1.addShowtime(showtime1)

CREATE OBJECT customer1 OF Customer("Ali", [])

CREATE OBJECT ticket1 OF Ticket(101, showtime1, seat1)

showtime1.bookTicket(ticket1)

DISPLAY showtime1.seatsAvailable()

showtime1.bookTicket(ticket1)

CREATE OBJECT ticket2 OF Ticket(showtime1, seat1)

showtime1.bookTicket(ticket2)

END

STOP

=====

Task 280: Warehouse Inventory A warehouse stores products supplied by different vendors. Each product belongs to a category and is kept in stock across several storage bins, and the total stock level for a product must always be the sum of what's in its bins — never a number set directly. Whenever stock moves in or out, a stock movement record is created to track the change, and a product should reject any outgoing movement that would push its stock below zero. A vendor supplies many products and has their own contact details and a reliability rating that updates based on how often their deliveries arrive late. Think about what a product, a storage bin, a stock movement, and a vendor each hold, how stock levels stay consistent, and how they combine to describe the warehouse's inventory.

CLASS Warehouse

ATTRIBUTES

warehouseName : Text
products : List<Product>
vendors : List<Vendor>

CONSTRUCTOR(n : Text, p : List<Product>, v : List<Vendor>)

SET warehouseName TO n
SET products TO p
SET vendors TO v

END CONSTRUCTOR

METHODS

addProduct(p : Product)
ADD p TO products
END METHOD

addVendor(v : Vendor)
ADD v TO vendors
END METHOD

END CLASS

CLASS Vendor

ATTRIBUTES

vendorID : Number
vendorName : Text
contact : Text
products : List<Product>

totalDeliveries : Number
lateDeliveries : Number

CONSTRUCTOR(n : Text, c : Text, p : List<Product>)
 SET vendorName TO n
 SET contact TO c
 SET products TO p
 SET totalDeliveries TO 0
 SET lateDeliveries TO 0
END CONSTRUCTOR

METHODS

addProduct(p : Product)
 ADD p TO products
END METHOD

recordDelivery(isLate : Boolean)
 SET totalDeliveries TO totalDeliveries + 1

 IF isLate EQUALS TRUE THEN
 SET lateDeliveries TO lateDeliveries + 1
 END IF
END METHOD

showReliability()
 DISPLAY totalDeliveries - lateDeliveries
END METHOD

END CLASS

CLASS Category

ATTRIBUTES

categoryID : Number
categoryName : Text

CONSTRUCTOR(n : Text)
 SET categoryName TO n
END CONSTRUCTOR

END CLASS

CLASS StorageBin

ATTRIBUTES

binID : Number
location : Text
quantity : Number

```
CONSTRUCTOR(l : Text, q : Number)
    SET location TO l
    SET quantity TO q
END CONSTRUCTOR
```

```
END CLASS
```

```
CLASS StockMovement
```

```
ATTRIBUTES
```

```
    movementID : Number
    movementType : Text
    quantity : Number
    date : Text
```

```
CONSTRUCTOR(t : Text, q : Number, d : Text)
    SET movementType TO t
    SET quantity TO q
    SET date TO d
END CONSTRUCTOR
```

```
END CLASS
```

```
CLASS Product
```

```
ATTRIBUTES
```

```
    productID : Number
    productName : Text
    category : Category
    vendor : Vendor
    bins : List<StorageBin>
    movements : List<StockMovement>
```

```
CONSTRUCTOR(n : Text, c : Category, v : Vendor, b : List<StorageBin>, m : List<StockMovement>)
    SET productName TO n
    SET category TO c
    SET vendor TO v
    SET bins TO b
    SET movements TO m
END CONSTRUCTOR
```

```
METHODS
```

```
    calcTotalStock()
        SET TOTAL TO 0

        REPEAT FROM i = 0 TO SIZE OF bins -1
            SET TOTAL TO TOTAL + bins[i].quantity
        END REPEAT
```



```
        DISPLAY TOTAL
    END METHOD
```

```
addMovement(m : StockMovement)
```

```
    SET CURRENT_STOCK TO 0
```

```
    REPEAT FROM I = 0 TO SIZE OF bins -1
```

```
        SET CURRENT_STOCK TO CURRENT_STOCK + bins[I].quantity
```

```
    END REPEAT
```

```
    IF m.movementType EQUALS "OUT" AND m.quantity GREATER THAN
    CURRENT_STOCK THEN
```

```
        DISPLAY "Movement rejected: stock cannot go below zero"
```

```
    ELSE
```

```
        ADD m TO movements
```

```
        DISPLAY "Movement added"
```

```
    END IF
```

```
END METHOD
```

```
END CLASS
```

```
START
```

```
    BEGIN
```

```
        CREATE OBJECT warehouse1 OF Warehouse("Main Warehouse", [], [])
```

```
        CREATE OBJECT vendor1 OF Vendor("Oman Supplies", "999999999", [])
```

```
        CREATE OBJECT category1 OF Category("Electronics")
```

```
        CREATE OBJECT bin1 OF StorageBin("A1", 50)
```

```
        CREATE OBJECT bin2 OF StorageBin("B1", 30)
```

```
        SET binList TO LIST [bin1, bin2]
```

```
        CREATE OBJECT product1 OF Product("Keyboard", category1, vendor1, binList, [])
```

```
        warehouse1.addVendor(vendor1)
```

```
        warehouse1.addProduct(product1)
```

```
        vendor1.addProduct(product1)
```

```
        product1.calcTotalStock()
```

```
        CREATE OBJECT move1 OF StockMovement("OUT", 40, "2026-07-05")
```

```
        product1.addMovement(move1)
```

```
CREATE OBJECT move2 OF StockMovement("OUT", 100, "2026-07-05")
product1.addMovement(move2)
```

```
vendor1.recordDelivery(TRUE)
vendor1.recordDelivery(FALSE)
```

```
vendor1.showReliability()
```

```
END
```

```
STOP
```

```
CLASS SchoolClass
```

```
ATTRIBUTES
```

```
    classID : Number
    teacher : Teacher
    subject : Subject
    students : List<Student>
    maxSize : Number
    scheduleTime : Text
```

```
CONSTRUCTOR(t : Teacher, s : Subject, st : List<Student>, max : Number, time : Text)
```

```
    SET teacher TO t
    SET subject TO s
    SET students TO st
    SET maxSize TO max
    SET scheduleTime TO time
END CONSTRUCTOR
```

```
METHODS
```

```
enrollStudent(student : Student)
```

```
    IF SIZE OF students GREATER THAN OR EQUAL TO maxSize THEN
        DISPLAY "Class is full"
```

```
    ELSE
        SET CONFLICT TO FALSE
```

```
    REPEAT FROM I = 0 TO SIZE OF student.classes - 1
        IF student.classes[I].scheduleTime EQUALS scheduleTime THEN
            SET CONFLICT TO TRUE
        END IF
    END REPEAT
```

```
    IF CONFLICT EQUALS TRUE THEN
```

```
        DISPLAY "Student already has class at this time"
    ELSE
        ADD student TO students
        ADD SELF TO student.classes
        DISPLAY "Student enrolled"
    END IF
END IF

END METHOD
```

```
END CLASS
```

```
CLASS Teacher
```

```
    ATTRIBUTES
        teacherID : Number
        teacherName : Text

    CONSTRUCTOR(n : Text)
        SET teacherName TO n
    END CONSTRUCTOR
```

```
END CLASS
```

```
CLASS Subject
```

```
    ATTRIBUTES
        subjectID : Number
        subjectName : Text
        syllabus : Text

    CONSTRUCTOR(n : Text, s : Text)
        SET subjectName TO n
        SET syllabus TO s
    END CONSTRUCTOR
```

```
END CLASS
```

```
CLASS Student
```

```
    ATTRIBUTES
        studentID : Number
        studentName : Text
        classes : List<SchoolClass>
        grades : List<Grade>

    CONSTRUCTOR(n : Text, c : List<SchoolClass>, g : List<Grade>)
        SET studentName TO n
        SET classes TO c
```

```
        SET grades TO g
    END CONSTRUCTOR
```

```
METHODS
```

```
addGrade(g : Grade)
    ADD g TO grades
END METHOD
```

```
calcAverage()
    SET TOTAL TO 0
    SET WEIGHT_TOTAL TO 0
```

```
    REPEAT FROM I = 0 TO SIZE OF grades -1
        SET TOTAL TO TOTAL + grades[I].score * grades[I].weight
        SET WEIGHT_TOTAL TO WEIGHT_TOTAL + grades[I].weight
    END REPEAT
```

```
    IF WEIGHT_TOTAL EQUALS 0 THEN
        DISPLAY 0
    ELSE
        DISPLAY TOTAL / WEIGHT_TOTAL
    END IF
```

```
END METHOD
```

```
END CLASS
```

```
CLASS Grade
```

```
    ATTRIBUTES
        assessmentName : Text
        score : Number
        weight : Number
```

```
    CONSTRUCTOR(a : Text, s : Number, w : Number)
        SET assessmentName TO a
        SET score TO s
        SET weight TO w
    END CONSTRUCTOR
```

```
END CLASS
```

```
START
```

```
    BEGIN
```

```
        CREATE OBJECT teacher1 OF Teacher("Mr. Ahmed")
```

```
        CREATE OBJECT subject1 OF Subject("Math", "Algebra and Geometry")
```

```
CREATE OBJECT class1 OF SchoolClass(teacher1, subject1, [], 2, "10:00")
```

```
CREATE OBJECT student1 OF Student("Said", [], [])
```

```
CREATE OBJECT student2 OF Student("Ali", [], [])
```

```
class1.enrollStudent(student1)
```

```
class1.enrollStudent(student2)
```

```
CREATE OBJECT grade1 OF Grade("Quiz", 80, 0.4)
```

```
CREATE OBJECT grade2 OF Grade("Final", 90, 0.6)
```

```
student1.addGrade(grade1)
```

```
student1.addGrade(grade2)
```

```
student1.calcAverage()
```

```
END
```

```
STOP
```